ECE 661: Computer Vision, Fall 2026

Policy Concerning Homework Submissions

Your Name:	En-Yu Fan

1. **Regarding the use of AI assistants:** I understand that my homework submissions will always start with a section titled **"AI Assistant Usage"** that will mention how much of my solution was generated by AI. For example, if I submitted a code file with, say, 100 lines of code and 30 of those were supplied by AI, this section will include a statement like "AI Usage: 30%". **In addition, if a function included in my code was generated entirely by AI, I'll make sure that the doc string for the function mentions that fact.**

2. I understand that, regardless of my usage of AI, the code I submit as my homework solution will be subject to plagiarism detection tools and compared against the code libraries at other web sites and the past solutions submitted for this class.

3. I understand that plagiarism detection may be carried out at any time whatsoever during the semester — including at the very end of the semester. I also understand that if I am found to have plagiarized in any of my homework solutions, my name would be submitted to the Dean of Students with a recommendation for expulsion from Purdue.

4. I also understand that if I am found to have plagiarized in *any* of my homework solutions, I will receive the failing grade F at the end of the semester regardless of my performance in all other homework assignments.



Your signature

ECE 661 Fall 2026: Homework 1

En-Yu Fan

fan474@purdue.edu

AI Assistant Usage

AI Usage: 45%

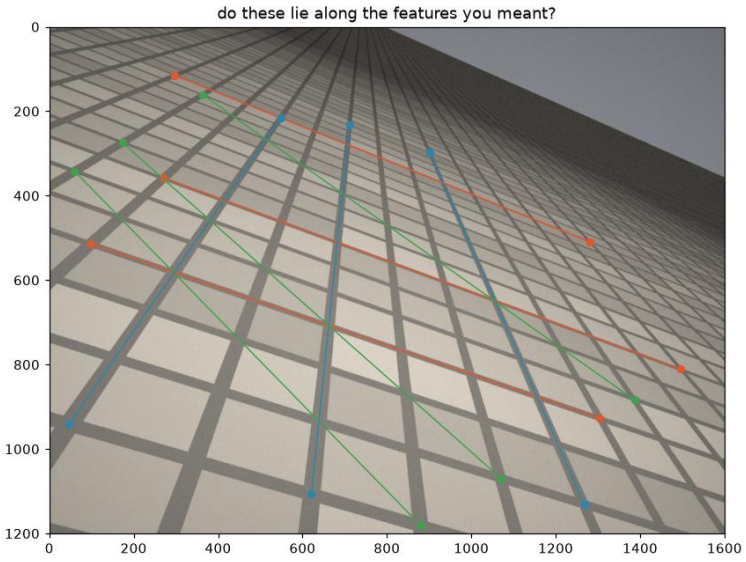
Task 1 (a)

Coordinate-reading Tool Used – GIMP

Recorded Coordinates (all up->down, left->right)

	Start	End
Tile rows	(297, 115)	(1281, 509)
	(270, 358)	(1496, 811)
	(99, 514)	(1304, 927)
Tile columns	(549, 215)	(47, 942)
	(711, 233)	(621, 1107)
	(901, 298)	(1268, 1132)
Tile diagonals	(364, 161)	(1387, 884)
	(173, 274)	(1071, 1070)
	(60, 342)	(881, 1182)

Point Overlay Check & Code Snippet



The recorded points align with intended tile-row, tile-column, and tile-diagonal features

```
RECORDED_TILES = {
    "A  tile rows":      [((297, 115), (1281, 509)),
                          ((270, 358), (1496, 811)),
                          ((99, 514), (1304, 927))],
    "B  tile columns":   [((549, 215), (47, 942)),
                          ((711, 233), (621, 1107)),
                          ((901, 298), (1268, 1132))],
    "C  tile diagonals": [((364, 161), (1387, 884)),
                          ((173, 274), (1071, 1070)),
                          ((60, 342), (881, 1182))],
}
```

Task 1(a) Code Snippet

Task 1 (b)

Prediction Before Computation

Before computation, I predicted that Family A (tile rows) would give the least reliable vanishing point. Since their projected lines appear closest to parallel in the image, their intersections should be far from the image. As a result, small errors in the recorded points can cause large changes in the estimated vanishing point.

Method

Following the procedure in the homework instruction, I made ***vanishing_point_estimates()*** to process one family at a time. It uses ***join()*** to construct three lines, then loops through the 3 line pairs. For each pair, ***meet()*** computes the intersection and ***dehom()*** converts it to Cartesian coordinates. Eventually, three estimates and their arithmetic mean are returned.

In ***task_1()***, I looped through the three families, using ***bounding_box_diagonal()*** to calculate each spread, and use ***np.linalg.norm()*** to calculate the distance from each mean VP to center point (800, 600). Finally, I compare the spreads and check whether my prediction held.

Vanishing-Point Results (in pixels)

Family	Estimate line 1&2	Estimate line 1&3	Estimate line 2&3	Spread	Mean VP	Distance from (800,600)
A	(8480.70, 3391.81)	(8392.70, 3356.57)	(8291.03, 3321.73)	202.20	(8388.14, 3356.70)	8073.37
B	(741.57, −63.88)	(741.68, −64.04)	(741.61, −64.22)	0.35	(741.62, −64.05)	666.61
C	(−1207.25, −949.47)	(−1191.12, −938.07)	(−1169.92, −916.38)	49.88	(−1189.43, −934.64)	2512.56

Prediction Check

Family A (tile rows) did have the largest spread at 202.20 pixels and farthest mean vanishing point from the image center. This is consistent with the observation that its nearly parallel lines make the intersection more sensitive to small point-selection errors.

Output & Code

```
A  tile rows                          B  tile columns                       C  tile diagonals
  VP-12: (8480.70, 3391.81)             VP-12: (741.57, -63.88)             VP-12: (-1207.25, -949.47)
  VP-13: (8392.70, 3356.57)             VP-13: (741.68, -64.04)             VP-13: (-1191.12, -938.07)
  VP-23: (8291.03, 3321.73)             VP-23: (741.61, -64.22)             VP-23: (-1169.92, -916.38)
  Spread: 202.20 pixels                 Spread: 0.35 pixels                 Spread: 49.88 pixels
  Mean VP: (8388.14, 3356.70)           Mean VP: (741.62, -64.05)           Mean VP: (-1189.43, -934.64)
  Distance to center: 8073.37 pixels    Distance to center: 666.61 pixels    Distance to center: 2512.56 pixels

Largest spread: A  tile rows
```

Task 1(b) output

```
574  def vanishing_point_estimates(pairs_of_points):
586      # build homogeneous lines from n pairs of points
587      lines = [join(p, q) for p, q in pairs_of_points]
588      # intersects the 3 pairs & dehomogenize each intersection -> 3 vanishing point estimates
589      estimates = []
590      for i, j in [(0, 1), (0, 2), (1, 2)]:
591          vp = dehom(meet(lines[i], lines[j]))
592          if vp is None:  # in cae of choosing an unsuitable img
593              raise ValueError(f"Line {i + 1} and {j + 1} have no finite VP")
594          estimates.append(vp)
595
596      estimates = np.asarray(estimates, dtype=float)
597      mean_vp = np.mean(estimates, axis=0)
598
599      return estimates, mean_vp
600
601  def task_1(img_name, recorded, output_task, img_c):
602      tiles = load_image(img_name)
603      ##  Look before you compute.
604      figure = show_recorded_points(tiles, recorded,
605                                    "do these lie along the features you meant?")
606      show_or_save(figure, f"{output_task}_recorded_points.png")
607
608      res = {}
609      img_center = img_c
610      print("\nTask 1(b)")
611      for name, pairs in recorded.items():
612          # VP estimates, spread, and distance to the image center.
613          estimates, mean_vp = vanishing_point_estimates(pairs)
614          spread = bounding_box_diagonal(estimates)
615          dist = float(np.linalg.norm(mean_vp - img_center))
616
617          # store results for later parts
618          res[name] = {
619              "estimates": estimates,
620              "spread": spread,
621              "mean_vp": mean_vp,
622              "distance": dist
623          }
624          # task 1-b output
625          print(f"\n{name}")
626          for (i, j), vp in zip([(1, 2), (1, 3), (2, 3)], estimates):
627              print(f"VP-{i}{j}: ({vp[0]:.2f}, {vp[1]:.2f})")
628
629          print(f"  Spread: {spread:.2f} pixels")
630          print(f"  Mean VP: ({mean_vp[0]:.2f}, {mean_vp[1]:.2f})")
631          print(f"  Distance to center: {dist:.2f} pixels")
632      # check if prediction held
633      worst_one = max(res, key=lambda name: res[name]["spread"])
634      print(f"\nLargest spread: {worst_one}")
```

Task 1(b) Code Snippet

Task 1(c)

Method

Following the TODO hint, I ordered the families by increasing spread and retrieved their mean vanishing points from the result obtained by 1(b) and used ***join()*** to construct the line through the two best-determined points. Finally, I calculated the perpendicular distance from the worst-determined mean VP to this line.

Order by spread (smallest -> largest)

Order	Family	Spread(pixels)
1	B: Tile columns	0.35
2	C: Tile diagonals	49.88
3	A: Tile rows	202.20

Discussion

The spread order from smallest to largest was B -> C -> A. Since A was the worst-determined family, I constructed the line through the mean VPs of B and C. The perpendicular distance from A's mean VP to this line was ***24.25*** pixels. This distance measures A's deviation from collinearity in image pixels.

Although a zero determinant can test exact collinearity, its magnitude is not meaningful since multiplying a homogeneous VP by 10^6 leaves the point unchanged while multiplying the determinant by 10^6. However, the perpendicular distance has interpretable pixel units and remains unchanged when the homogeneous line coefficients are rescaled.

The 24.25-pixel residual is small relative to the observed VP variation and therefore supports the claim that the three VPs share a vanishing line. However, a large residual that remained after careful re-recording would have falsified the claim.

Output & Code

```
Task 1(c): ordered by spread (smallest first)
  B  tile columns: 0.35 pixels
  C  tile diagonals: 49.88 pixels
  A  tile rows: 202.20 pixels
Line goes through: B  tile columns & C  tile diagonals
Collinearity residual: 24.25 pixels
```

Task 1(c) output

```
    ##  TODO (c): order the families by spread; compute the perpendicular
    ##            distance from the WORST-determined mean vanishing point to
    ##            the line through the other two mean vanishing points.
    ordered_family = sorted(res, key=lambda name: res[name]["spread"])
    good_1, good_2, worst_one = ordered_family

    # use mean VPs from 1(b).
    vp_1 = res[good_1]["mean_vp"]
    vp_2 = res[good_2]["mean_vp"]
    vp_worst = res[worst_one]["mean_vp"]
    # line through 2 better VPs
    vanishing_line = join(vp_1, vp_2)

    # perpendicular distance from the worst VP to this line, ax+by+c=0
    line_norm = np.linalg.norm(vanishing_line[:2])
    # just in case accidentlly enter 2 same families
    if line_norm == 0:
        raise ValueError("2 better families's VPS not defining a valid finite line")
    # dist between point & the line
    residual = float(abs(np.dot(vanishing_line, hom(vp_worst))) / line_norm)

    print("\nTask 1(c)")
    for name in ordered_family:
        print(f"  {name}: {res[name]['spread']:.2f} pixels")

    print(f"Line goes through: {good_1} & {good_2}")
    print(f"Collinearity residual: {residual:.2f} pixels")
```

Task 1(c) Code Snippet

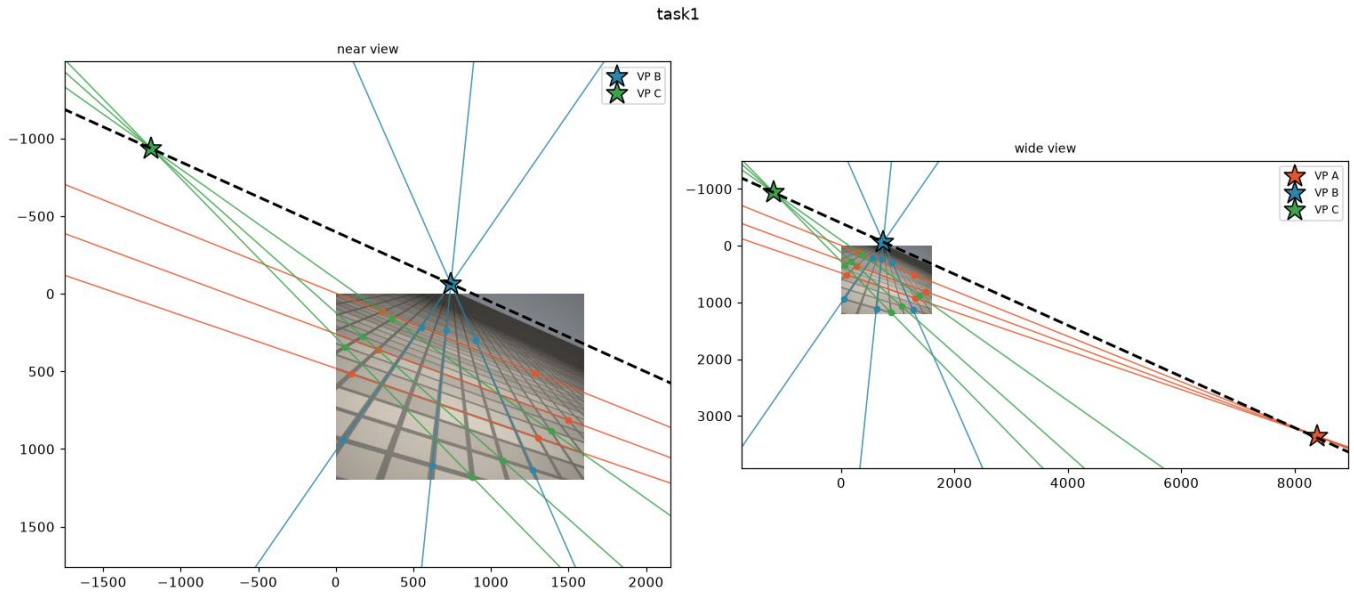
Task 1(d)

Method

Following the TODO hint, I retrieved the mean VPs of two best-determined families (B and C) from the results of 1(b), and the vanishing line from 1(c). The normalized line coefficients are “a: 0.4110”, “b: -0.9116”, “c: -363.1961”.

In ***two_panel_figure()*** for plotting the original image, I used ***np.linalg.norm()*** to sort 3 VPs by their distances from the image center. The near-view axis limits are modified to include the image and the two nearer VPs with an additional margin, while the wide view included all three VPs. In addition, I use ***axis.axline()*** to extend each of the nine recorded lines across the entire panel, along with some space arrangement to avoid covering VP A. Finally, I plotted the original image, recorded points, nine extended lines, three mean VPs, and the vanishing line using the two-panel format.

Figure



Near and wide views of the recorded points, nine extended lines, mean VPs, and vanishing line. The black dashed line is the vanishing line through mean VPs of B and C.

Discussion

A Cartesian implementation must special-case two distinct image lines that are exactly parallel. Their intersection is an ideal point at infinity, so it cannot be represented by finite Cartesian coordinates. A vanishing point that is only far away is still finite and does not require this special treatment.

The cross-product handles with no special case because it represents the intersection of parallel lines as a homogeneous point with a third coordinate of zero.

As a result, test is still needed in ***dehom()*** before dividing by that coordinate. This shows that the difficulty lies in converting an ideal point to Cartesian coordinates, rather than in computing the intersection with the cross product.

Output & Code

```
Task 1(d): vanishing line, with a^2 + b^2 = 1
  [a:0.4110, b:-0.9116, c:-363.1961]
```

Task 1(d) Output

```
    #near_x = (-0.35 * width, 1.6 * width)
    #near_y = (1.15 * height, -0.4 * height)
    ########################################################
    # make the view include image & two VPs nearest to center.
    img_center = np.array([width / 2.0, height / 2.0])
    near_vps = sorted(vanishing_points,
                      key=lambda v: np.linalg.norm(v - img_center))[:2] # first 2

    near_vx = [v[0] for v in near_vps]
    near_vy = [v[1] for v in near_vps]
    # inclusive range for margin & most outside VPs.
    near_x = (min(-margin, min(near_vx) - margin),
              max(width + margin, max(near_vx) + margin))
    near_y = (max(height + margin, max(near_vy) + margin),
              min(-margin, min(near_vy) - margin))
    ########################################################
```

```
  #t = np.linspace(-1, 9, 2)
  #P = A + t[:, None] * (B - A)
  #axis.plot(P[:,0], P[:,1], color=color, linewidth=1.0, alpha=0.85)

  ############################################################
  # Extend the line across the entire panel.
  axis.axline(A, B, color=color, linewidth=1.0, alpha=0.85)
  ############################################################
```

```
        ########################################################
        #axis.legend(loc="lower right", fontsize=8)
        axis.legend(loc="upper right", fontsize=8) # to prevent overlapping VP A
        ########################################################
```

two_panel_figure() modification

```
    ##  TODO (d): the vanishing line as the join of the two BEST-determined
    ##            vanishing points, then two_panel_figure().

    # use original family order -> colors match the recorded points.
    mean_vps = [res[name]["mean_vp"] for name in recorded]

    # normalize the vanishing line computed in 1(c)
    a, b, c = vanishing_line / line_norm

    print("\nTask 1(d)")
    print(f"  [a:{a:.4f}, b:{b:.4f}, c:{c:.4f}]")

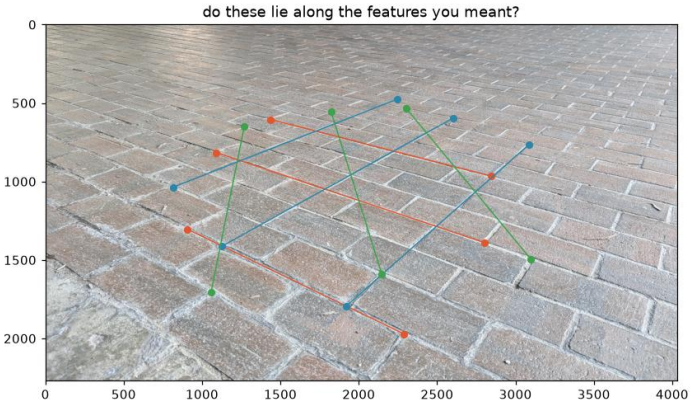
    figure = two_panel_figure(
        tiles, recorded,
        mean_vps, vanishing_line,
        output_task
    )
    show_or_save(figure, f"{output_task}_vline.png")
```

Task 1(d) Code Snippet

Task 1(e)

My photograph & Code & recorded coordinate table





I basically do the same thing to this photograph, including the code. But due to its original size(4032x2268), I modified the image center to (2016, 1134) temporarily

```
task_1(
    "tiles.jpg",
    RECORDED_TILES,
    "task1",
    np.array([800.0, 600.0])
)
print("################################################")
print("################# Next 1(e) ####################")
print("################################################")
task_1(
    "bricks.jpg",
    RECORDED_OWN,
    "task1_e",
    np.array([2016.0, 1134.0])
)
```

I modify the structure of task1(), passing the recorded points sets & corresponding image center to enable faster printing without uncommenting something.

	Start	End
Tile rows	(1436, 605)	(2847, 966)
	(1091, 816)	(2801, 1389)
	(908, 1305)	(2290, 1972)
Tile columns	(2243, 476)	(815, 1036)
	(2603, 595)	(1126, 1413)
	(3088, 763)	(1921, 1793)
Tile diagonals	(2305, 536)	(3098, 1495)
	(1823, 554)	(2147, 1590)
	(1269, 651)	(1058, 1703)

Prediction and vanishing-point estimates

Before computing the vanishing points, I expected the brick-column family to be the least reliable, which is always because they seem to be most parallel among the 3 families.

The results are below, the largest spread is obtained by B and consistent with my prediction.

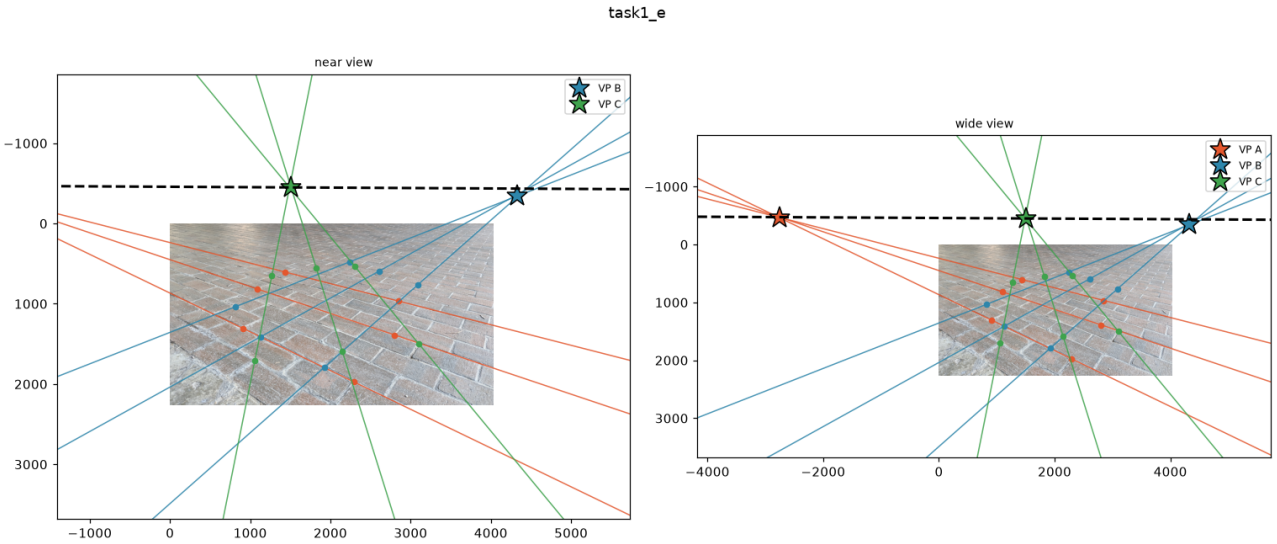
Family	Estimate line 1&2	Estimate line 1&3	Estimate line 2&3	Spread	Mean VP	Distance from (2016,1134)
A	(−2685.68, −449.52)	(−2774.25, −472.18)	(-2821.82, -495.14)	143.58	(-2760.59, -472.28)	5039.44
B	(4212.32, -296.28)	(4348.83, -349.82)	(4415.96, -409.06)	232.78	(4325.70, -351.72)	2746.29
C	(1520.77, -412.40)	(1489.80, -449.85)	(1497.32, -487.36)	81.11	(1502.63, -449.87)	1664.99

Spread and collinearity residual

```
Task 1(c): ordered by spread (smallest first)
  C  tile diagonals: 81.11 pixels
  A  tile rows: 143.58 pixels
  B  tile columns: 232.78 pixels
Line goes through: C  tile diagonals & A  tile rows
Collinearity residual: 83.31 pixels
```

The families in increasing order of spread are C -> A -> B, so I use the mean VPs of C and A to define the vanishing line. The perpendicular distance from the mean VP of B to this line is 83.31 pixels.

Vanishing-line result




```
Task 1(e): vanishing line, with a^2 + b^2 = 1
  [a:0.0053, b:-1.0000, c:-457.7626]
```

The normalized vanishing-line coefficients are around (0.0053, −1.0000, −457.7626). All 3 VPs are outside the 4032 × 2268 image frame. Two panels are still required for showing all three vanishing points.

Discussion

The collinearity residual for my photograph is 83.31 pixels, which is much larger than that of ***tiles.jpg*** (24.25 pixels). One possible lens-related cause is radial lens distortion from my phone camera. It can make straight brick edges appear slightly curved, especially near the image boundaries. Lines recorded from different parts of the photograph may therefore fail to meet at exactly consistent vanishing points.

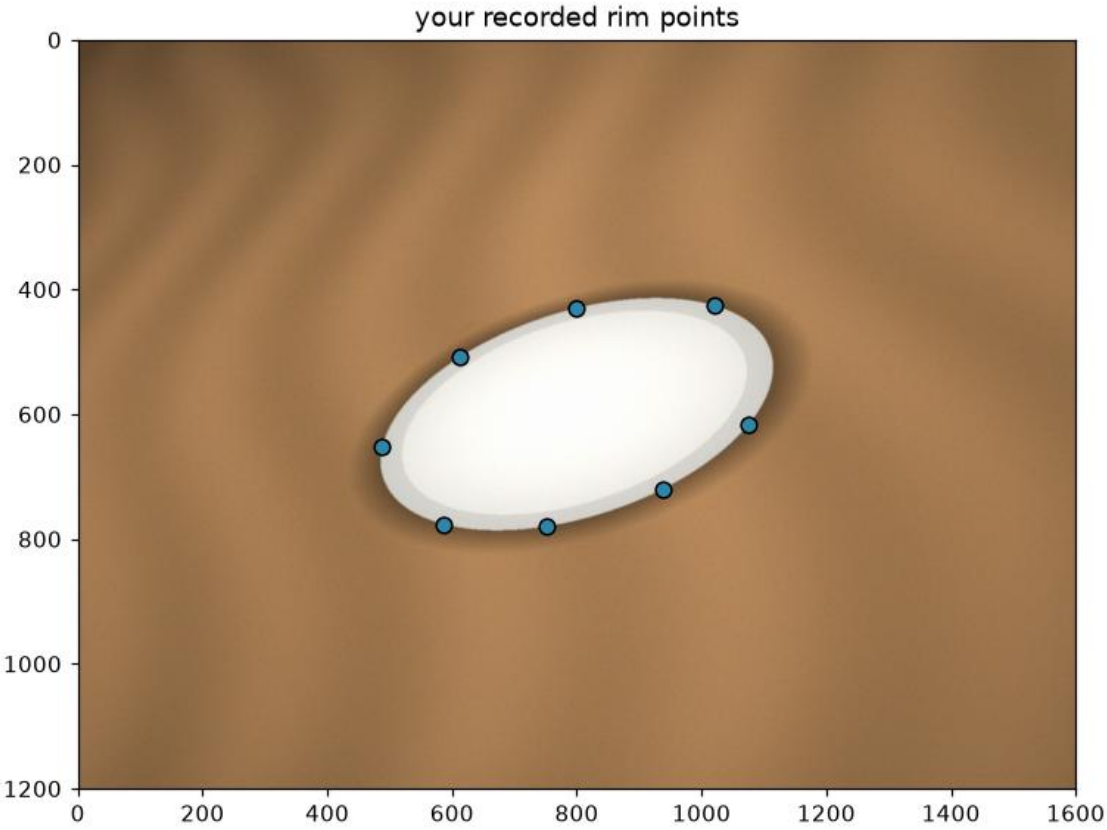
To distinguish lens distortion from recording error, I would check whether the deviations follow a consistent spatial pattern. Lens distortion should produce larger, systematic deviations near the image edges, while point-recording errors should be more random.

Task 2 (a)

Recorded Coordinates (start from the leftmost point, clockwise)

[488, 651]	[613, 508]	[800, 430]	[1021, 426]
[1076, 616]	[939, 721]	[752, 780]	[586, 777]

Point Overlay Check & Code Snippet



Point Overlay

```
Implement with Codex
##  TODO (a): replace with the eight points YOU recorded around the OUTER edge
##            of the plate.  Do not mix in the inner ring where the glaze
##            changes shade; that is a second, smaller circle.

RECORDED_RIM = np.array([[488, 651], [613, 508], [800, 430], [1021, 426],
                         [1076, 616], [939, 721], [752, 780], [586, 777]], dtype=float)
```

Task 2(a) Code Snippet

Task 2 (b)

Method

The ***fit_conic()*** formed an 8x6 design matrix ***A*** from the 8 recorded rim points. For each point ***(x,y)***, a row of ***A*** is constructed as ***[x^2, xy, y^2, x, y, 1]*** then applied with ***numpy.linalg.svd()***. I use the last row of Vt, which is the right singular vector corresponding to the smallest singular value, as the six conic coefficients. The coefficients are then convert into symmetric conic matrix ***C*** by ***coefficients_to_matrix()***, which is divided by its Frobenius norm to fix its scale ($||C||_F$ =1).

Finally, I converted each recorded point to homogeneous form by ***hom()*** and evaluated $|x_i^T C x_i|$. The largest value among the eight points was reported as the maximum algebraic residual.

Results

The fitted conic matrix, normalized so that $||C||_F=1$, is:

$$C = \begin{bmatrix} -4.45947750 \times 10^{-7} & -3.04297065 \times 10^{-7} & 5.39478431 \times 10^{-4} \\ -3.04297065 \times 10^{-7} & -1.27367637 \times 10^{-6} & 1.00827417 \times 10^{-3} \\ 5.39478431 \times 10^{-4} & 1.00827417 \times 10^{-3} & -9.99998692 \times 10^{-1} \end{bmatrix}$$

The maximum algebraic residual over the 8 recorded points is 2.673255×10^{-4}

```
Maximum algebraic residual: 2.673255e-04
```

Discussion

Fixing the scale of ***C*** is necessary because multiplying ***C*** by any nonzero scalar represents the same conic. However, $x_i^T C x_i$ changes by the same scalar. Without ($||C||_F$ =1), the reported residual could be made arbitrarily large or small without changing the fitted conic.

The reported number is the largest absolute algebraic residual among the 8 recorded points. It measures the worst disagreement with the normalized implicit conic equation. It is not a distance in pixels because it is the output of a quadratic equation, not the shortest Euclidean distance from a point to the fitted conic. Its value also depends on the local gradient of the conic.

Output & Code

```
Task 2(b): fitted C with Frobenius norm 1
[[-4.45947750e-07 -3.04297065e-07  5.39478431e-04]
 [-3.04297065e-07 -1.27367637e-06  1.00827417e-03]
 [ 5.39478431e-04  1.00827417e-03 -9.99998692e-01]]
Maximum algebraic residual: 2.673255e-04
```

Task 2(b) output

```python
def fit_conic(points):
    """
        This implementation was written with assistance from an AI assistant.
    """
    points = np.asarray(points, dtype=float)
    x = points[:, 0]
    y = points[:, 1]

    # each point contributes one row to the conic design matrix.
    # shape: (8, 6)
    A = np.column_stack((x**2, x*y, y**2, x, y, np.ones_like(x)))

    # coef. vec = "right singular vector" in smallest singular value of A
    # which is the last row of Vt
    _, _, Vt = np.linalg.svd(A, full_matrices=False)
    coefficients = Vt[-1]

    # convert 6 coefficients into symmetric conic matrix.
    # shape: (3, 3)
    conic_matrix = coefficients_to_matrix(coefficients)

    # fix the otherwise arbitrary scale of the conic matrix.
    frobenius_norm = np.linalg.norm(conic_matrix, ord="fro")
    if frobenius_norm == 0:
        raise ValueError("conic fit produced a zero matrix")

    return conic_matrix / frobenius_norm # C with Frobenius norm 1
```

```python
    ##  TODO (b): fit C and report max_i |x_i^T C x_i| with ||C||_F = 1.
    ##            Expect something of order 1e-3 to 1e-4.  If you get order 1,
    ##            you have taken the wrong singular vector.
    recorded_conic = fit_conic(RECORDED_RIM)

    # evaluate the normalized conic equation at all eight recorded points.
    residuals = [
        abs(hom(point) @ recorded_conic @ hom(point)) # x^T C x
        for point in RECORDED_RIM
    ]
    max_residual = max(residuals)

    print("\nTask 2(b)")
    print(recorded_conic)
    print(f"Maximum algebraic residual: {max_residual:.6e}")
```

Task 2(b) Code Snippet

Task 2 (c)

Method

I reconstructed the coefficient vector from the symmetric matrix ***C***, including multiplying 2 for the off-diagonal terms. I then divided all 6 coefficients by the norm of the 3 quadratic coefficients ***[a,b,c]***, so that $a^2+b^2+c^2=1$. Finally, I computed the discriminant and classified the fitted conic from its sign.

Output results

Quantity	a	b	c	$a^2+b^2+c^2$	b^2-4ac	Classification
Value	-0.3012	-0.4111	−0.8604	1.0000	−0.8677	Ellipse

Discussion

My normalized discriminant is -0.8677. Since it is negative, the fitted conic is an ellipse. It is also close to the expected -0.86, rather than -2, 0, or other positive values. This shows that a general conic was fitted, the normalization used only ***[a, b, c]***, and the recorded points were sufficiently distributed around the rim. Despite the fact that the plate is a circle in the world, the fitted image conic does not satisfy the circle constraints, since the ***b≠0***, and ***a≠c***.

A general camera view produces a projective transformation of the plate's plane, which mixes the coordinates and therefore also mixes the individual entries of ***C***, so relations such as ***b=0***, and ***a=c*** are not generally preserved. However, its boundary remained a nondegenerate conic. Point incidence and tangency were also preserved.

Output & Code

```
Task 2(c): normalize quadratic coefficients
  a = -0.3012, b = -0.4111, c = -0.8604
  a^2 + b^2 + c^2 = 1.0000
  Discriminant = -0.8677
  Conic Type: ellipse
```

Task 2(c) output

```
    ##  TODO (c): rescale so that a^2 + b^2 + c^2 = 1, report b^2 - 4ac and
    ##            classify.  Expect a value near -0.86 on plate.jpg.
    # Recover [a, b, c, d, e, f] from the symmetric conic matrix.
    coeffs = np.array([
        recorded_conic[0, 0], 2 * recorded_conic[0, 1],
        recorded_conic[1, 1], 2 * recorded_conic[0, 2],
        2 * recorded_conic[1, 2], recorded_conic[2, 2]
    ], dtype=float)

    # rescale using only the 3 quadratic coefficients (a,b,c) so that a^2 + b^2 + c^2 = 1
    abc_norm = np.linalg.norm(coeffs[:3])
    scaled_coeffs = coeffs / abc_norm
    a, b, c = scaled_coeffs[:3]

    # classify the conic by the discriminant.
    discriminant = b**2 - 4 * a * c

    if np.isclose(discriminant, 0.0, atol=1e-10):
        conic_type = "parabola"
    elif discriminant < 0:
        conic_type = "ellipse"
    else:
        conic_type = "hyperbola"

    print("\nTask 2(c)")
    print(f"  a = {a:.4f}, b = {b:.4f}, c = {c:.4f}")
    print(f"  a^2 + b^2 + c^2 = {a**2 + b**2 + c**2:.4f}")
    print(f"  Discriminant = {discriminant:.4f}")
    print(f"  Conic Type: {conic_type}")
```

Task 2(c) code snippet

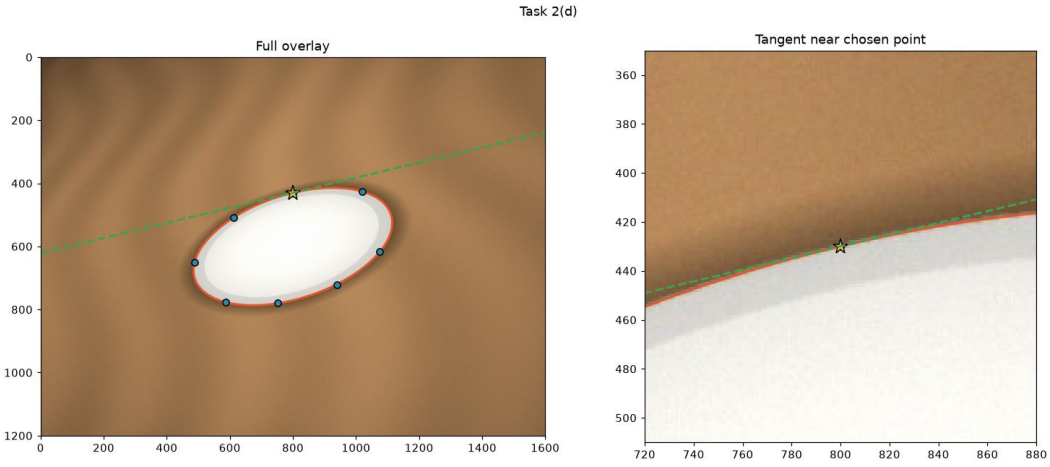
Task 2 (d)

Method

I choose the point ***(800,430)*** and convert it to homogeneous coordinates, since its tangent is not close to vertical and can therefore be drawn reliably with ***draw_line_across()***. The tangent line is computed as ***l = Cp*** using the fitted conic matrix from 2(b). I then used ***draw_line_across()*** and ***draw_conic()*** to overlay the fitted conic, the 8 recorded points, and the tangent on the image. Then I used a second panel to show enlarged region around the chosen point ***p***. Finally, I evaluate $\boldsymbol{p^TCp}$ and $\boldsymbol{l^T p}$ using the same point and conic matrix.

Results & Figure

```
Task 2(d): tangent check
  Point p = (800, 430)
  p^T C p = 1.688458e-05
  l^T p   = 1.688458e-05
```



Discussion

The full overlay I get has the fitted conic following the outer rim of the plate. In the enlarged view, the tangent line touches the conic at the chosen point without crossing it. This visual check supports that the fitted ***C*** is consistent with the plate rim.

The two computed values are

$$p^T Cp = \ell^T p = 1.688458 \times 10^{-5}$$

They must agree because ***l = Cp***, which leads to

$$\ell^T p = (Cp)^T p = p^T C^T p$$

Since ***C*** is symmetric, we can know

$$\ell^T p = p^T Cp$$

This agreement does not show that the conic fit is right, since this equality follows automatically from ***l = Cp*** and the symmetry of ***C***. It would still hold for an incorrectly fitted symmetric matrix. The quality of the fit is instead checked by the image overlay, such as “the conic should follow the plate rim”, and “the tangent should touch the curve at the selected point without crossing” as the instruction says.

Output & Code

```python
    ##  TODO (d): the tangent at one recorded point, l = C p.  Overlay the
    ##            conic, the points and the tangent, then zoom in on p and
    ##            confirm the line touches the curve there without crossing it.
    # choose 1 recorded rim point & compute its tangent line l = C p.
    p = RECORDED_RIM[2] # 3rd point (800, 430), since it's relatively near horizontal
    p_hom = hom(p)
    tg_line = recorded_conic @ p_hom

    # overlay the fitted conic, recorded points, and tangent line.
    figure, axes = plt.subplots(1, 2, figsize=(14, 6))
    height, width = plate.shape[:2]
    zoom_radius = 80

    for axis in axes:
        axis.imshow(plate)

        # draw the fitted conic and 8 recorded points.
        draw_conic(axis, recorded_conic, plate.shape)
        axis.plot(
            RECORDED_RIM[:, 0], RECORDED_RIM[:, 1], "o",
            color="#2e86ab", markersize=6, markeredgecolor="k"
        )

        # Mark the selected point and draw its tangent.
        axis.plot(
            p[0], p[1], "*",
            color="#f4c542", markersize=14, markeredgecolor="k"
        )
        draw_line_across(
            axis, tg_line, (0, width),
            color="#3fa34d", linewidth=2
        )

        axis.set_aspect("equal")
```

```python
    # image view.
    axes[0].set_xlim(0, width)
    axes[0].set_ylim(height, 0)
    axes[0].set_title("Full overlay")
    # closer view around chosen point.
    axes[1].set_xlim(p[0] - zoom_radius, p[0] + zoom_radius)
    axes[1].set_ylim(p[1] + zoom_radius, p[1] - zoom_radius)
    axes[1].set_title("Tangent near chosen point")

    figure.suptitle("Task 2(d)")
    plt.tight_layout()
    show_or_save(figure, "task2_tangent.png")

    # conic equation & line equation at chosen point.
    pCp = float(p_hom @ recorded_conic @ p_hom)
    line_at_p = float(tg_line @ p_hom)

    print("\nTask 2(d): tangent check")
    print(f"  Point p = ({p[0]:.0f}, {p[1]:.0f})")
    print(f"  p^T C p = {pCp:.6e}")
    print(f"  l^T p   = {line_at_p:.6e}")
```

Task 2(d) code snippet

Bonus (a)

Method

The aiming line was constructed by joining the origin with the unit direction point ***(cos α, sin α)***. Then I computed the endpoint angles of each segment with ***atan2()***, mapped them to ***[0°, 360°)***, and returned the shorter angular interval by ***subtended_interval()***, including the case where it crosses 0°. Eventually, return the shorter angular interval.

The target boundary rays were then evaluated at ***y = 4*** to check whether the crossed the armour. Finally, the aiming line was met with ***l∞***, and the resulting ideal point was normalized for reporting.

Results & Discussion

The homogeneous aiming line obtained by joining $[0, 0, 1]^T$ and $[\cos \alpha, \sin \alpha, 1]^T$ is: $\boldsymbol{l(\alpha) = [-\sin \alpha, \cos \alpha, 0]^T}$. As a result, the ***α*** appears in the first two entries.

The angular intervals were:

Segment	Angular interval
Target	69.4440° to 82.8750°
Armour	38.6598° to 84.2894°

In addition to the target interval being contained within the armour interval, the target rays cross ***y = 4*** over ***x = 0.5000 to 1.5000***, which is inside the armour range ***x = 0.4000 to 5.0000***. Since the armour is at ***y = 4*** and the target is at ***y = 8***, an upward ray reaches the armour first and no direct shot exists.

When meeting at aiming line with $\boldsymbol{l_\infty = [0, 0, 1]^T}$, it gives $\boldsymbol{l(\alpha) \times l_\infty = [\cos \alpha, \sin \alpha, 0]^T}$, up to a nonzero scale. This is the ideal point representing the aiming direction.

Output & Code

```
Bonus (a)
  Target interval: 69.4440, 82.8750 degrees
  Armour interval: 38.6598, 84.2894 degrees
  Target interval contained in armour interval: True
  Target rays cross y = 4.0              over x = [0.5000, 1.5000]
  Armour covers x = [0.4000, 5.0000]
  Armour is met first: True
  Direct shot exists: False

Sample alpha: 69.4440 degrees
  Aiming line: [-0.93632918  0.35112344  0.        ]
  Ideal point: [ 0.35112344  0.93632918 -0.        ]
```

Bonus(a) output

```python
def aiming_line(alpha_in_degrees):
    '''TODO (a): the aiming line, as a homogeneous 3-vector.

    It is the join of the origin and a point one unit away in the direction
    alpha.  Say in your report where alpha ends up in the three entries, and
    why the third entry comes out the way it does.
    '''
    # convert the input angle into radian for NumPy.
    alpha = np.deg2rad(alpha_in_degrees)

    # join player & a unit point in aiming direction.
    player = np.array([0.0, 0.0]) # origin
    direction_point = np.array([np.cos(alpha), np.sin(alpha)])
    return join(player, direction_point)

def subtended_interval(segment):
    '''TODO (a): the angular interval a segment subtends at the origin.

    Return the SMALLER of the two arcs bounded by the two endpoint directions.
    Watch the case where that arc straddles the zero direction.
    '''
    # find each endpoint direction
    angles = np.array([
        np.degrees(np.arctan2(point[1], point[0]))
        for point in segment
    ])
    # represent it in degree [0, 360)
    angles = np.mod(angles, 360.0)
    start, end = np.sort(angles)
    # If the direct interval > 180 degrees, use the arc across 0 to get the smaller arc
    if end - start > 180.0:
        return end, start + 360.0

    return start, end
```

```python
def bonus():
    ##  TODO (a): report both intervals, and show that for every angle in the
    ##            target's interval, the ray crosses y = 4 inside the armour.
    target_interval = subtended_interval(TARGET)
    armour_interval = subtended_interval(ARMOUR)
    print("\nBonus (a)")
    print(f"  Target interval: {target_interval[0]:.4f}, {target_interval[1]:.4f} degrees")
    print(f"  Armour interval: {armour_interval[0]:.4f}, {armour_interval[1]:.4f} degrees")
    if_contained = (
        target_interval[0] >= armour_interval[0]
        and target_interval[1] <= armour_interval[1]
    )
    print(f"  Target interval contained in armour interval: {if_contained}")

    # check where 2 boundary rays of the target interval cross y = 4.
    target_y = TARGET[0][1] # = 8
    armour_y = ARMOUR[0][1] # = 4
    boundary_angles = np.deg2rad(np.array(target_interval))
    crossing_x = armour_y / np.tan(boundary_angles)
    crossing_interval = (np.min(crossing_x), np.max(crossing_x))

    armour_x = sorted([ARMOUR[0][0], ARMOUR[1][0]]) # interval in order
    crossings_inside = (
        crossing_interval[0] >= armour_x[0]
        and crossing_interval[1] <= armour_x[1]
    ) # check if the ray crosses y = 4 inside the armour

    # the target rays travel "upward" through y = 4 before reaching y = 8.
    upward_rays = np.all(np.sin(boundary_angles) > 0.0)
    armour_met_first = (crossings_inside
                        and upward_rays and 0.0 < armour_y < target_y
    )

    print(f"  Target rays cross y = {armour_y:.1f} "
          f"              over x = [{crossing_interval[0]:.4f}, {crossing_interval[1]:.4f}]")
    print(f"  Armour covers x = [{armour_x[0]:.4f}, {armour_x[1]:.4f}]")
    print(f"  Armour is met first: {armour_met_first}")
    print(f"  Direct shot exists: {not armour_met_first}")
```

```python
    ##  TODO (a): meet the aiming line with l_infinity = [0,0,1]^T and say what
    ##            you get.  This is the first ideal point you will compute.
    sample_alpha = target_interval[0]
    sample_line = aiming_line(sample_alpha)
    line_at_infinity = np.array([0.0, 0.0, 1.0])
    ideal_point = meet(sample_line, line_at_infinity)

    # Normalize the first two entries to print clearer
    ideal_point = ideal_point / np.linalg.norm(ideal_point[:2])
    print(f"\nSample alpha: {sample_alpha:.4f} degrees")
    print(f"  Aiming line: {sample_line}")
    print(f"  Ideal point: {ideal_point}")
```

Bonus (b)

Method

I first made ***strikes_segment()*** compute the meet ***q*** between aiming line and the line containing the segment, then checked if ***q*** falls between the segment endpoints. An ideal meet is rejected because it is not a finite point on the segment. Since the meet already lies on the segment's supporting line, its coordinates are compared with the endpoint bounds, using a small tolerance for rounding error.

strikes_segment() then optionally applies the ray-direction test $\boldsymbol{d^T q > 0}$. In bonus(), it is first called at α = 250° with the direction test disabled to get the naive result. The meet ***q*** and $\boldsymbol{d^T q}$ are computed for report later, than ***strikes_segment(*** is called again with the direction test enabled to obtain the corrected result.

Results & Discussion

At $\boldsymbol{\alpha = 250°}$, the meet with the target line was $\boldsymbol{q = (2.9118, 8.0000)}$. Since this point lies between the target endpoints, the naive test reported a hit. A homogeneous line has no direction, so 250° and 70° describe the same infinite line even though they point in opposite directions.

However, the direction was $\boldsymbol{d = (-0.3420, -0.9397)^T}$, and $\boldsymbol{d^T q = -8.5134}$. The negative result shows that ***q*** lies behind player along the same undirected aiming line. The missing condition is thus $\boldsymbol{d^T q > 0}$, this is strict because $\boldsymbol{d^T q = 0}$ would give no positive travel distance from the player. After adding this condition, the corrected test showed that the target was not hit.

Output & Code

```
Bonus (b)
  Meet q: (2.9118, 8.0000)
  Naive line test reports a hit: True
  Direction d: (-0.3420, -0.9397)
  d^T q: -8.5134
  Corrected ray test reports a hit: False
```

Bonus(b) output

```python
def strikes_segment(segment, alpha_in_degrees, use_direction_test=True):
    """
        This implementation was written with assistance from an AI assistant.
    """
    # build 2 infinite lines and find the homogeneous intersection.
    segment_line = join(segment[0], segment[1])
    shot_line = aiming_line(alpha_in_degrees)
    q = dehom(meet(segment_line, shot_line))

    # parallel lines meet at an ideal point, not on a finite segment.
    if q is None:
        return False

    # the meet lies on the segment's line, so test whether it is within the bounds.
    tolerance = 1e-9 # in case some rounding error makes q fall just outside the segment
    lower_bound = np.minimum(segment[0], segment[1]) - tolerance
    upper_bound = np.maximum(segment[0], segment[1]) + tolerance
    # check if q's (x,y) lies within the bounding range
    falls_between = np.all(q >= lower_bound) and np.all(q <= upper_bound)

    if not falls_between:
        return False

    # check direction
    if use_direction_test:
        alpha = np.deg2rad(alpha_in_degrees)
        direction = np.array([np.cos(alpha), np.sin(alpha)])
        # reject an intersection that lies behind player
        if direction @ q <= 0.0:
            return False

    return True
```

```python
    ##  TODO (b): the naive test at alpha = 250, then the corrected one.
    test_alpha = 250.0
    # the requested naive test with undirected aiming line.
    naive_hit = strikes_segment(TARGET, test_alpha, use_direction_test=False)

    # recompute the meet to report its position and direction test
    target_line = join(TARGET[0], TARGET[1])
    q = dehom(meet(target_line, aiming_line(test_alpha)))

    alpha = np.deg2rad(test_alpha)
    direction = np.array([np.cos(alpha), np.sin(alpha)])
    direction_test = direction @ q

    # the actual test
    corrected_hit = strikes_segment(TARGET, test_alpha, use_direction_test=True)
    print("\nBonus (b)")
    print(f"  Meet q: ({q[0]:.4f}, {q[1]:.4f})")
    print(f"  Naive line test reports a hit: {naive_hit}")
    print(f"  Direction d: ({direction[0]:.4f}, {direction[1]:.4f})")
    print(f"  d^T q: {direction_test:.4f}")
    print(f"  Corrected ray test reports a hit: {corrected_hit}")
```

Bonus(b) code snippet

Bonus (c)

Method

I represented the wall ***x = 7*** by the normalized homogeneous line ***l = [1, 0, −7]^T***. ***reflection_in_line()*** used this wall line to construct ***M = I − 2ñl^T***. I then computed ***M²*** and used ***np.allclose()*** to compare it with the 3 × 3 identity matrix.

Results & Discussion

For the wall x = 7, the reflection matrix was

M = [[−1, 0, 14],

[0, 1, 0],

[0, 0, 1]].

Squaring this matrix gave

M² = [[1, 0, 0],

[0, 1, 0],

[0, 0, 1]] = I,

and the numerical comparison returned True. This had to be true because reflecting a point across the same line twice returns it to its original position.

Output & Code

```
Bonus (c)
  Wall line l: [ 1.  0. -7.]
  Reflection matrix M:
[[-1.  0. 14.]
 [ 0.  1.  0.]
 [ 0.  0.  1.]]
  M^2:
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
  M^2 equals I: True
```

Bonus(c) output

```
416     ##  TODO (c): M for the wall x = 7; check that M @ M is the identity.
417     wall_line = np.array([1.0, 0.0, -7.0])# x = 7
418     reflection_M = reflection_in_line(wall_line)
419     reflection_squared = reflection_M @ reflection_M
420     is_I = np.allclose(reflection_squared, np.eye(3))
421
422     print("\nBonus (c)")
423     print(f"  Wall line l: {wall_line}")
424     print("  Reflection matrix M:")
425     print(reflection_M)
426     print("  M^2:")
427     print(reflection_squared)
428     print(f"  M^2 equals I: {is_I}")
```

Bonus(c) code snippet

Bonus (d)

Method

I reflected both target endpoints with ***reflection_M*** and averaged the reflected points to obtain the mirror-image center. The aiming angle was computed with ***atan2()***, and ***aiming_line()*** constructed the incoming path.

I traced this path by meeting it with armour and wall lines, while ***strikes_segment()*** checked whether it hit the finite segments. The incoming direction was represented by its ideal point and reflected by ***reflection_M***. I then constructed the return line through the wall hit point in the reflected direction, met it with the target line, and checked if the landing point fell within the target bounds

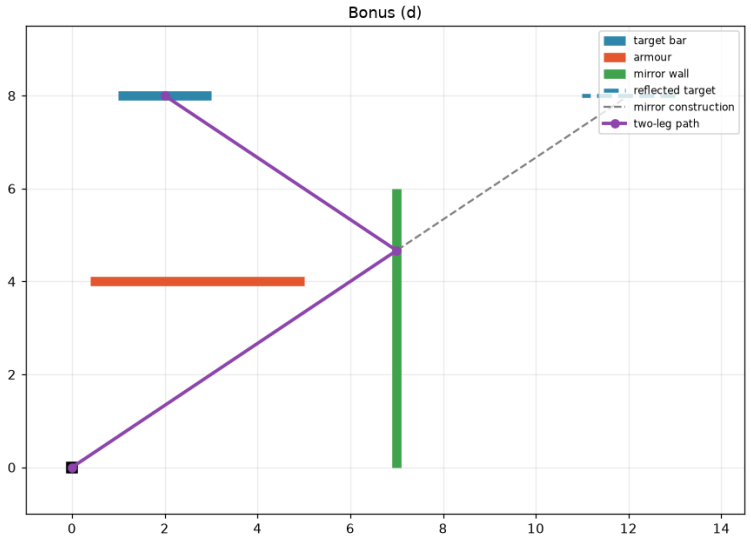
Results & Discussion

```
Bonus (d)
  Reflected target endpoints: (13.0000, 8.0000) and (11.0000, 8.0000)
  Reflected target center: (12.0000, 8.0000)
  Aiming angle alpha: 33.6901 degrees
  Incoming ideal point: [0.8321, 0.5547, -0.0000]
  Crossing at armour height: (6.0000, 4.0000)
  Clears armour: True
  Wall hit: (7.0000, 4.6667)
  Hits wall segment: True
  Reflected ideal point: [-0.8321, 0.5547, 0.0000]
  Reflected direction: (-0.8321, 0.5547)
  Return-leg landing point: (2.0000, 8.0000)
  Return leg lands on target: True
```

Bonus(d) output

The reflected target centre was ***(12.0000, 8.0000)***, giving an aiming angle of ***33.6901°***. The incoming path crossed ***y = 4*** at ***x = 6.0000***, which is beyond the armour endpoint at ***x = 5.0000***, so it cleared the armour. It then hit the wall at ***(7.0000, 4.6667)***, within the wall's y-range from 0 to 6.

Applying ***reflection_M*** to the incoming ideal point changed the direction to ***(−0.8321, 0.5547)***. Its third coordinate was zero, so the translation part of the matrix did **not** affect the direction. The return leg landed at ***(2.0000, 8.0000)***, which lies on the target bar between ***x = 1*** and ***x = 3***.



Mirror-image construction and the traced two-leg path. The incoming path clears the armour and strikes the wall at ***(7.0000, 4.6667)***, while the reflected path lands on the target at ***(2.0000, 8.0000)***.

Code

```
    ##  TODO (d): reflect the target bar in the wall, aim at the mirror image,
    ##            then trace the ray and check every leg of it.  Plot the path
    ##            on top of draw_arena().

    # reflect both target endpoints across the wall.
    reflect_target = tuple(dehom(reflection_M @ hom(point))
                           for point in TARGET)
    mirror_center = (reflect_target[0] + reflect_target[1]) / 2.0

    # aim at mirror & compute angle
    # aim from player(0, 0) at the center of the reflected target & get shooting line.
    ricochet_alpha = np.degrees(np.arctan2(mirror_center[1], mirror_center[0]))# (y, x)
    shoot_line = aiming_line(ricochet_alpha)

    # confirm that it clears the armour.
    # find where the incoming path crosses armour line.
    arm_line = join(ARMOUR[0], ARMOUR[1])
    arm_crossing = dehom(meet(shoot_line, arm_line)) # intersect y=4
    clear_arm = not strikes_segment(ARMOUR, ricochet_alpha)

    # find where the incoming path strikes the finite wall segment.
    wall_hit = dehom(meet(shoot_line, wall_line))
    hits_wall = strikes_segment(WALL, ricochet_alpha)

    # use ideal point from (a) to represent the aiming direction.
    aiming_ideal = meet(shoot_line, line_at_infinity)
    aiming_ideal = (aiming_ideal / np.linalg.norm(aiming_ideal[:2]))

    # reflect direction
    # multiply reflection M with incoming direction as a homogeneous vec
    reflected_ideal = reflection_M @ aiming_ideal
    reflected_direction = (reflected_ideal[:2] / np.linalg.norm(reflected_ideal[:2]))

    # return line from the wall hit and reflected direction.
    return_line = join(wall_hit,  wall_hit + reflected_direction)
    target_line = join(TARGET[0], TARGET[1])
    landing_point = dehom(meet(return_line, target_line))

    target_x = sorted([TARGET[0][0], TARGET[1][0]])
    land_on_target = (target_x[0] <= landing_point[0] <= target_x[1])

    print("\nBonus (d)")
    print(f"  Reflected target endpoints: "
          f"({reflect_target[0][0]:.4f}, "
          f"{reflect_target[0][1]:.4f}) and "
          f"({reflect_target[1][0]:.4f}, "
          f"{reflect_target[1][1]:.4f})"
    )
    print(f"  Reflected target center: "
          f"({mirror_center[0]:.4f}, {mirror_center[1]:.4f})"
    )
    print(f"  Aiming angle alpha: {ricochet_alpha:.4f} degrees")
    print(f"  Incoming ideal point: "
          f"[{aiming_ideal[0]:.4f}, "
          f"{aiming_ideal[1]:.4f}, "
          f"{aiming_ideal[2]:.4f}]"
    )
    print(f"  Crossing at armour height: "
          f"({arm_crossing[0]:.4f}, "
          f"{arm_crossing[1]:.4f})"
    )
    print(f"  Clears armour: {clear_arm}")
    print(f"  Wall hit: "
          f"({wall_hit[0]:.4f}, {wall_hit[1]:.4f})"
    )
    print(f"  Hits wall segment: {hits_wall}")
    print(f"  Reflected ideal point: "
          f"[{reflected_ideal[0]:.4f}, "
          f"{reflected_ideal[1]:.4f}, "
          f"{reflected_ideal[2]:.4f}]"
    )
    print(f"  Reflected direction: "
          f"({reflected_direction[0]:.4f}, "
          f"{reflected_direction[1]:.4f})"
    )
    print(f"  Return-leg landing point: "
          f"({landing_point[0]:.4f}, "
          f"{landing_point[1]:.4f})"
    )
    print(f"  Return leg lands on target: {land_on_target}")

    ##########
    figure, axis = plt.subplots(figsize=(8, 6))
    draw_arena(axis)

    # Show the reflected target and the straight mirror construction.
    axis.plot([reflect_target[0][0], reflect_target[1][0]],
              [reflect_target[0][1], reflect_target[1][1]],
              "--", color="#2e86ab", linewidth=3,
              label="reflected target"
    )
    axis.plot([0.0, mirror_center[0]],
              [0.0, mirror_center[1]],
              "--", color="gray", linewidth=1.5,
              label="mirror construction"
    )

    # Draw the actual two-leg path through the wall hit.
    axis.plot([0.0, wall_hit[0], landing_point[0]],
              [0.0, wall_hit[1], landing_point[1]],
              color="#8e44ad", linewidth=2.5, marker="o",
              label="two-leg path"
    )

    axis.set_title("Bonus (d)")
    axis.legend(loc="upper right", fontsize=8)
    plt.tight_layout()
    show_or_save(figure, "Bonus(d)_ricochet_path.png")
```

Bonus(d) code snippet